```php
<?php
// S - SPOOFING MITIGATION: Start session to securely track the authenticated state
session_start();

// Using 'connect.php' based on your folder structure
include 'connect.php';

$message = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $user = $_POST['username'];
    $pass = $_POST['password'];

    // D - DENIAL OF SERVICE MITIGATION: Basic Rate Limiting
    // Initialize login attempts if not set
    if (!isset($_SESSION['login_attempts'])) {
        $_SESSION['login_attempts'] = 0;
    }

    // Lock the form after 5 failed attempts for 60 seconds
    if ($_SESSION['login_attempts'] >= 5) {
        if (time() - $_SESSION['last_attempt_time'] < 60) {
            // R - REPUDIATION MITIGATION: Log the brute-force attempt
            error_log("Security Audit: Rate limit exceeded for IP: " . $_SERVER['REMOTE_ADDR']);
            die("Account locked due to too many failed attempts. Try again in 60 seconds.");
        } else {
            // Timeout expired, reset attempts
            $_SESSION['login_attempts'] = 0;
        }
    }

    // I - INFO DISCLOSURE & E - ELEVATION OF PRIVILEGE MITIGATION:
    // Fetch ONLY the user record by username. We grab the 'role' here to enforce access later.
    $stmt = $conn->prepare("SELECT id, username, password, role FROM accounts WHERE username = ?");
    $stmt->bind_param("s", $user);
    $stmt->execute();
    $result = $stmt->get_result();
```

```php
// I - INFO DISCLOSURE & E - ELEVATION OF PRIVILEGE MITIGATION:
// Fetch ONLY the user record by username. We grab the 'role' here to enforce access later.
$stmt = $conn->prepare("SELECT id, username, password, role FROM accounts WHERE username = ?");
$stmt->bind_param("s", $user);
$stmt->execute();
$result = $stmt->get_result();

if ($row = $result->fetch_assoc()) {

    // I - INFO DISCLOSURE MITIGATION: Secure Password Verification
    // Verify the raw POST password against the stored bcrypt hash
    if (password_verify($pass, $row['password'])) {

        // Reset failed attempts upon successful login
        $_SESSION['login_attempts'] = 0;

        // S & E - SPOOFING & ELEVATION: Bind identity and role to the server-side session
        $_SESSION['user_id'] = $row['id'];
        $_SESSION['username'] = $row['username'];
        $_SESSION['role'] = $row['role'];

        // T - TAMPERING MITIGATION: Regenerate the session ID to prevent Session Fixation attacks
        session_regenerate_id(true);

        $message = "Access Granted: Welcome, " . htmlspecialchars($row['username']) . " (Role: " . htmlspecialchars($row['role']) . ")";

        // R - REPUDIATION MITIGATION: Audit log the success
        error_log("Security Audit: Successful login for user - " . $user);

    } else {
        // Failed password match
        $_SESSION['login_attempts']++;
        $_SESSION['last_attempt_time'] = time();
        $message = "Access Denied: Invalid credentials.";
        error_log("Security Audit: Failed login (bad password) for user - " . $user);
```

```php
            $message = "Access Granted: Welcome, " . htmlspecialchars($row['username']) . " (Role: " . htmlspecialchars($row['role']) . ")";

            // R - REPUDIATION MITIGATION: Audit log the success
            error_log("Security Audit: Successful login for user - " . $user);

        } else {
            // Failed password match
            $_SESSION['login_attempts']++;
            $_SESSION['last_attempt_time'] = time();
            $message = "Access Denied: Invalid credentials.";
            error_log("Security Audit: Failed login (bad password) for user - " . $user);
        }
    } else {
        // Failed user match. (We use the EXACT same error message so attackers can't guess usernames)
        $_SESSION['login_attempts']++;
        $_SESSION['last_attempt_time'] = time();
        $message = "Access Denied: Invalid credentials.";
        error_log("Security Audit: Failed login (user not found) for - " . $user);
    }

    $stmt->close();
}
?>
```

System Login

Access Denied: Invalid credentials.

Username:

Password:

Login